

JAVA 17 OCP · YDS STUDY SERIES

Unit 14 · I/O — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge sekiz adet **OCP tarzı özgün çalışma sorusu** içerir; sorular gerçek sınavdan alınmamıştır. Önerilen süre 25–30 dakikadır; istersen 1–4 ve 5–8 olarak iki oturuma böl. Path sorularında textual operation ile file-system access'i, stream sorularında byte ile character hierarchy'sini önce ayır.

Sorular

Soru 1

Odak: `Path` immutability ve normalization

Aşağıdaki programın çıktısı nedir?

```
import java.nio.file.Path;

public class PathCount {
    public static void main(String[] args) {
        Path path = Path.of("a", "b", "..", "c");
        System.out.print(path.getNameCount() + ":"
            + path.normalize().getNameCount());
    }
}
```

- A. 4:2
- B. 2:2
- C. 4:4
- D. Kod derlenmez.

Soru 2

Odak: `InputStream.read()`

Aşağıdaki programın çıktısı nedir?

```
import java.io.ByteArrayInputStream;

public class ReadBytes {
    public static void main(String[] args) throws Exception {
        byte[] data = {65, 66};
        try (var input = new ByteArrayInputStream(data)) {
            System.out.print(input.read() + ":"
                + input.read() + ":" + input.read());
        }
    }
}
```

- A. A:B:null
- B. 65:66:0
- C. 65:66:-1
- D. Kod derlenmez; `read()` checked exception declare eder.

Soru 3**Odak:** `normalize()` ve `toRealPath()`

Hangisi doğrudur?

- A. `normalize()` target'ın varlığını doğrulamak için file system'e erişir.
- B. `toRealPath()` yalnız textual `.` ve `..` temizliği yapar; target'ın varlığı önemli değildir.
- C. `normalize()` textual ve yeni bir `Path` result'ı üretir; `toRealPath()` file system'e erişebilir ve target yoksa `IOException` oluşturabilir.
- D. Her iki method da original `Path` object'ini yerinde değiştirir.

Soru 4**Odak:** Stream ve `Files` API tuzaklarıAşağıdaki ifadelerden **hangi ikisi doğrudur?**

- A. `Files.walk()` resource-backed bir `Stream<Path>` döndürebileceği için try-with-resources ile kapatılmalıdır.
- B. `Files.copy(sourceDirectory, targetDirectory)` bütün alt ağacı recursive olarak kopyalar.
- C. `Reader/Writer` character data; `InputStream/OutputStream` byte data içindir.
- D. `InputStream.available()` bütün remaining byte sayısını her zaman kesin olarak verir.
- E. `Files.walk()` default olarak bütün symbolic link'leri izler.

Soru 5**Odak:** DeserializationOrdinary bir class `Serializable` implement ediyor ve ilk non-serializable superclass'ı erişilebilir no-arg constructor'a sahip. Deserialization sırasında hangisi doğrudur?

- A. Serializable class'ın bütün constructor'ları normal object creation'daki gibi çalışır.
- B. Hiçbir superclass constructor'ı çalışmaz.
- C. İlk non-serializable superclass'ın no-arg constructor'ı çalışır; serializable class'ın constructor'ı çalışmaz.
- D. `transient` field'ların constructor değerleri mutlaka geri yüklenir.

Soru 6**Odak:** English → Turkish / YDSAşağıdaki cümleyi doğal Türkçeye çevir; `because` ve modal passive yapısını belirt:

`Files.walk()` should be used within try-with-resources because the returned stream may hold open directory handles.

Soru 7

Odak: Tampon kapasitesi ile gerçekten okunan miktar

Aşağıdaki tam program için **tek doğru** sonuç hangisidir?

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class PartialBuffer {
    public static void main(String[] args) throws IOException {
        var source = new ByteArrayInputStream(new byte[]{65, 66});
        var target = new ByteArrayOutputStream();
        byte[] buffer = new byte[4];
        int count = source.read(buffer);
        target.write(buffer, 0, count);
        System.out.print(count + ":" + target.size() + ":"
            + target.toString(StandardCharsets.US_ASCII));
    }
}
```

- A. 2:2:AB
- B. 4:4:AB ve iki görünmez karakter
- C. Derlenmez; `write()` uzunluk alamaz.
- D. `IndexOutOfBoundsException` oluşur.

Karşı örnek: `write(buffer, 0, count)` yerine `write(buffer)` kullanılırsa hedefte kaç bayt olur? Cevaba bakmadan gerekçeni yaz.

Soru 8

Odak: `Path` işlemi ile dosyanın varlığı

Unix biçimli varsayılan dosya sistemi kabul edildiğinde aşağıdaki tam program nasıl sonuçlanır? **Tek seçenek seç.**

```
import java.nio.file.Path;

public class MixedPaths {
    public static void main(String[] args) {
        var base = Path.of("/work/java");
        var other = Path.of("notes.txt");
        System.out.print(base.relativize(other));
    }
}
```

- A. `../../notes.txt` yazdırır.
- B. `notes.txt` yazdırır.
- C. Derlenmez; `relativize()` checked exception bildirir.
- D. Derlenir; çalışma zamanında `IllegalArgumentException` oluşur.

Dil aktarımı: “The call fails because one path is absolute whereas the other is relative.” cümlesinde neden ve karşıtlık bağlaçlarını ayır.

Cevaplar ve açıklamalar

Soru 1 — A

- **A doğru:** Original relative path dört name element içerir: `a`, `b`, `..`, `c`. `normalize()` textual olarak `b/..` çiftini kaldırır; normalized result iki elementlidir.
- **B yanlış:** `Path` immutable olduğundan original `path` iki elemente dönüşmez.
- **C yanlış:** `normalize()` bu örnekte reducible `b/..` bölümünü kaldırır.
- **D yanlış:** Kod Java 17 `Path` API'sini geçerli biçimde kullanır.

Soru 2 — C

- **A yanlış:** `InputStream.read()` character veya `null` değil `int` döndürür.
- **B yanlış:** End of stream değeri `0` değildir.
- **C doğru:** İlk iki çağrı unsigned byte değerleri olan `65` ve `66`yı, üçüncü çağrı EOF göstergesi `-1`i döndürür.
- **D yanlış:** `ByteArrayInputStream.read()` checked exception bildirmez; resource kapanışından gelebilecek checked exception ise `main` imzasında declare edilmiştir. Program derlenir.

Soru 3 — C

- **A yanlış:** `normalize()` pure textual operation'dır.
- **B yanlış:** `toRealPath()` gerçek file-system target'ını çözer ve varlık kontrolü yapar.
- **C doğru:** İki method arasındaki temel sınav ayrımı textual dönüşüm ile file-system access'tir.
- **D yanlış:** `Path` immutable'dır; method'lar original object'i mutasyona uğratmaz.

Soru 4 — A ve C

- **A doğru:** Directory stream backing resource'u açık kalabileceğinden returned stream kapatılmalıdır.
- **B yanlış:** Tek `Files.copy()` çağrısı directory tree'yi recursive copy etmez.
- **C doğru:** `Reader/Writer` character, `InputStream/OutputStream` byte hierarchy'sidir.
- **D yanlış:** `available()` blocking olmadan okunabileceği tahmin edilen miktarı bildirir; total remaining length garantisi değildir.
- **E yanlış:** `Files.walk()` symbolic link'leri yalnız `FileVisitOption.FOLLOW_LINKS` verilirse izler.

Soru 5 — C

- **A yanlış:** Ordinary serializable class'ın constructor ve instance initializer'ları deserialization sırasında çalışmaz.
- **B yanlış:** İlk non-serializable superclass'ın uygun no-arg constructor'ı çalışır.
- **C doğru:** Serializable bölüm stream'den restore edilirken non-serializable superclass state'i constructor ile kurulur. Bu constructor da kendi normal superclass constructor zincirini başlatabilir.
- **D yanlış:** `transient` instance field serialize edilmez; default değerle kalabilir ve subclass constructor'ı onu yeniden initialize etmez.

Bu kural ordinary class içindir; serializable record deserialization sırasında canonical constructor'ını çalıştıran önemli bir istisnadır.

Soru 6 — Örnek çeviri

“`Files.walk()` try-with-resources içinde kullanılmalıdır; çünkü döndürülen stream açık directory handle'ları tutabilir.”

`because` doğrudan neden bildirir. `should be used` modal passive (`should + be + V3`) bir öneri verir; `returned` ise “döndürülen” anlamındaki reduced passive relative clause'dur.

Soru 7 — A

- **A doğru:** Kaynakta yalnız iki bayt vardır; `read()` 2 döndürür. Yalnız `count` kadar yazıldığı için hedef de iki bayttır: `2:2:AB`.
- **B yanlış:** Tamponun kapasitesi 4 olması, dört bayt okunduğu anlamına gelmez.
- **C yanlış:** `write(byte[], int, int)` geçerli bir overload'dur; program derlenir.
- **D yanlış:** Başlangıç 0 ve uzunluk 2, dört elemanlı tamponun sınırları içindedir.

Karşı örnek: `write(buffer)` dört bayt yazar; son iki bayt dizinin başlangıç değeri olan 0 olur. Dosya kopyalamada her yinelemede yalnız gerçekte okunan miktarı yaz.

Soru 8 — D

- **D doğru:** Unix varsayımında `base` mutlak, `other` görelidir; bu iki biçim doğrudan `relativize()` edilemez. Kod derlenir, çalışma zamanında `IllegalArgumentException` oluşur.
- **A ve B yanlış:** Sonuç yolu hesaplanmadan yol türleri uyumsuzluğu nedeniyle işlem durur.
- **C yanlış:** `relativize()` bu işlem için checked `IOException` bildirmez; dosyanın varlığını da sorgulamaz.

Çeviri: “Yollardan biri mutlak, diğeri görelidir olduğundan çağrı başarısız olur.” `because` neden, `whereas` iki yolun özelliği arasındaki karşıtlığı bildirir.